

프로젝트 : NumPy 기반 영화 평점 분석

IMDb Top 1000 영화 데이터를 활용한 분석 및 시각화

팀 명 : 열정걸즈

팀 원 : 한다예, 김혜민(CTO)

날 짜 : 2025.11.26

프로젝트 개요

목표: NumPy를 활용하여 영화 데이터를 분석하고, 장르/연도별 평점 트렌드를 파악합니다.

사용 데이터: [IMDB Top 1000 CSV](#)

NumPy 기반 영화 평점 분석 프로젝트

태그	Colab	Matplotlib	PyCharm	Python	github	팀프로젝트
날짜	@2025년 11월 12일 → 2025년 11월 26일					



프로젝트명: NumPy 기반 영화 평점 분석 프로젝트

- 언어: Python
- 환경: PyCharm (로컬 IDE), Google Colab (클라우드 실습)
- 버전관리: Git, GitHub
- 라이브러리: NumPy, Matplotlib

• 주요 기능:

1. NumPy 배열을 활용해 영화 데이터 분석
2. 장르별/연도별 평균 평점 분석
3. 최고 평점 영화 확인
4. 시각화를 통해 트렌드 분석 가능
5. 프로젝트 확장: 감독/배우별 평점 분석, 조건 필터링

👥 팀 구성 & 역할

• 김혜민 (CTO)

- 프로젝트 구조 설계, Step1~2 완료, GitHub 관리
- 프로젝트 확장 기능 구현 및 시각화
 - 특정 감독별 평균 평점 분석
 - 특정 배우별 평균 평점 비교
 - 특정 조건 영화 필터링

• 한다예 (팀원)

- Step3~5 기능 구현
 - Step3: 평점이 높은 영화 찾기
 - Step4: 장르별 평균 평점 분석
 - Step5: 연도별 평점 변화 분석

역할 분담 요약:

Step	담당
1~2	김혜민
3~5	한다예
프로젝트 확장 & 시각화	김혜민

📁 프로젝트 구조

```
NumPy_Movie_Project/
├── 01_data/
│   └── IMDB_top_1000.csv      # 원본 CSV 데이터
├── 02_scripts/
│   ├── step1.py              # 데이터 준비 + 전처리
│   ├── step2.py              # 기본 데이터 탐색
│   ├── step3.py              # 평점 높은 영화 찾기
│   ├── step4.py              # 장르별 평균 평점 분석
│   └── step5.py              # 연도별 평균 평점 분석
├── 03_outputs/
│   ├── results/              # Step 실행 결과 파일
│   └── visualization/        # Step PNG, PDF 파일
├── 04_extension/             # 프로젝트 확장 분석
│   ├── ext_prepare.py         # 확장 분석용 데이터 준비
│   ├── ext_step2_director.py  # 특정 감독별 평균 평점 계산
│   ├── ext_step3_star.py      # 특정 배우별 평균 평점 계산
│   └── ext_step4_filter.py     # 특정 조건 충족 영화 필터링
├── .gitignore
├── README.md
└── requirements.txt
```

Step 1: 데이터 준비

목표

- CSV 데이터를 순수 Python + NumPy로 읽기
- Title, Genre, Rate 컬럼만 선택
- Released_Year 는 Title 에서 슬라이싱 추출
- 결측값 제거 후 NumPy 배열로 변환
- 데이터 크기와 데이터 형태 확인

접근 방법

1. 안전 CSV 파싱 함수 정의

- 큰 따옴표 안에 있는 심표는 문자열로 유지
- 심표 기준으로 컬럼 분리

2. 헤더에서 필요한 컬럼 인덱스 추출

- Title, Genre, Rate 컬럼 위치 확인

3. 데이터 줄 반복

- 안전 파싱 후 필요한 컬럼 추출
- Title 앞 숫자 제거 (1. The Shawshank Redemption → The Shawshank Redemption)
- 결측값 제거
- Title에서 Released_Year 추출
- 데이터 타입 변환 (Rate: float, Released_Year: int)

4. NumPy 배열 변환 및 확인

5. NumPy 배열 바이너리 파일로 저장

- .npy 파일로 저장 → 이후 np.load 로 쉽게 불러올 수 있음

구현 코드

```
import numpy as np

# csv 파일 경로
csv_path = "../01_data/IMDB top 1000.csv"

# csv 파싱 함수 정의
# 큰 따옴표 안에 있는 쉼표와 바깥에 있는 쉼표 구분
def parse_csv_line(line):
    row = []      # 한 줄을 분리한 문자열 담을 리스트
    current = ""   # 현재 문자열 누적
    inside_quotes = False # 큰 따옴표 안에 있는지 여부 체크
    for char in line:
        if char == '"' and not inside_quotes: # 큰 따옴표 시작
            inside_quotes = True
        elif char == '"' and inside_quotes: # 큰 따옴표 종료
            inside_quotes = False
        elif char == ',' and not inside_quotes: # 따옴표 밖에 쉼표
            row.append(current) # 여태 누적된 문자열 리스트 안에 담기
            current = ""      # 누적 초기화
        else:
            current += char    # 문자 누적
    row.append(current)      # 마지막 문자열 리스트 안에 담기
    return row              # for문 밖에서 반환

# csv 파일 읽기
with open(csv_path, "r", encoding="utf-8") as f:
    lines = f.read().splitlines() # 줄 단위 리스트 생성

# 헤더와 데이터 분리 후
header = parse_csv_line(lines[0]) # 파싱 헤더 추출
data_lines = lines[1:]          # 데이터 줄만 따로 저장
print(header)
```

```
# 영화 제목, 장르, 평점 컬럼 인덱스 반환
title_idx = header.index("Title") # 1
genre_idx = header.index("Genre") # 4
rate_idx = header.index("Rate")   # 5

# 데이터 전처리 리스트 준비
processed_data = [] # 최종 NumPy 배열로 변환할 데이터 담을 리스트

# 데이터 줄 반복 처리
for line in data_lines:
    row = parse_csv_line(line) # 파싱하여 컬럼 분리

    # row 길이가 필요한 컬럼 중 가장 큰 인덱스보다 작으면 해당 컬럼이 없다는 의미
    if len(row) <= max(title_idx, genre_idx, rate_idx):
        continue # 없다면, 건너 뛴다. 다음 줄을 처리하러 for문 처음으로 돌아감.

    # 컬럼 값 가져오기 및 공백 제거
    Num_title = row[title_idx].strip()
    genre = row[genre_idx].strip()
    rate = row[rate_idx].strip()

    # Title 앞에 있는 숫자 제거
    title = Num_title.split(',')[1].strip() if ',' in Num_title else Num_title

    # 결측값 제거 : 빈 문자열이 있으면 해당 행 제외
    if title == "" or genre == "" or rate == "":
        continue

    # Title에서 Released_Year 추출
    # Title 마지막에 () 형태가 있어야만 슬라이싱
    released_year = None
    if '(' in title and title[-1] == ')':
        released_year = title[title.rfind('(')+1 : title.rfind(')')]
    else:
        # 괄호가 없으면 임의로 None 처리
        released_year = None

# Rate와 Released_Year 타입 변환 후 리스트에 추가
try:
    processed_data.append([title, genre, float(rate), int(released_year)])
except ValueError:
    # 변환 불가능한 값이 있으면 건너뛰기
    continue
```

```
# NumPy 배열로 변환
# dtype=object 사용 : 문자열 + 숫자가 혼합된 배열
data_array = np.array(processed_data, dtype=object)

# 결과 확인
print("NumPy 배열 형태의 데이터 출력: ", data_array)
print("\n데이터 크기(shape) 확인: ", data_array.shape)

# NumPy 배열 바이너리 파일로 저장
np.save("../04_outputs/data_array.npy", data_array)
```

parse_csv_line 함수 코드 설명

함수: `parse_csv_line(line)`

목적

- CSV 한 줄(`line`)을 쉼표 기준으로 컬럼으로 분리
- 단, 큰따옴표 안의 쉼표는 데이터로 취급 (예: `"Action, Adventure, Drama"`)

코드와 설명

```
row = []
```

- 설명:** 한 줄에서 추출한 컬럼들을 담은 빈 리스트 생성
- 예: CSV 줄 `"1,Action, Drama,8.9"` → 최종적으로 `['1', 'Action, Drama', '8.9']` 형태로 저장

```
current = ""
```

- 설명:** 현재 컬럼 문자열을 누적할 변수
- CSV 줄을 한 문자씩 읽으면서, 쉼표가 나오기 전까지의 문자열을 `current`에 모음

```
inside_quotes = False
```

- 설명:** 큰따옴표 안에 있는지 여부 체크
- `False` → 현재 큰따옴표 밖
- `True` → 현재 큰따옴표 안 (쉼표가 나와도 컬럼 구분 아님)

```
for char in line:
```

- 설명:** 한 줄의 문자열을 한 글자씩 순회
- CSV 처리 시, **큰따옴표 안의 쉼표와 밖의 쉼표를 구분**해야 하므로 문자 단위 반복 필요

큰따옴표 시작 / 종료 확인

```
if char == '"' and not inside_quotes:
    inside_quotes = True
elif char == '"' and inside_quotes:
    inside_quotes = False
```

- 설명:**
 - `char == '"' and not inside_quotes` → 큰따옴표 시작
 - `inside_quotes`를 `True` 로 바뀌서 이후 쉼표는 컬럼 구분이 아님
 - `char == '"' and inside_quotes` → 큰따옴표 종료
 - `inside_quotes`를 `False` 로 바뀌서 이후 쉼표는 컬럼 구분

예시:

```
"Action, Adventure, Drama"
```

- 처음 `"` → `inside_quotes = True`
- `"Action, Adventure, Drama"` → 쉼표 있어도 `current`에 누적
- 마지막 `"` → `inside_quotes = False`

쉼표 처리

```
elif char == ',' and not inside_quotes:
    row.append(current)
    current = ""
```

- 설명:**
 - 현재 문자가 쉼표이고, **큰따옴표 밖이라면**
 - 지금까지 누적이한 `current` 를 `row` 리스트에 컬럼으로 추가
 - `current` 초기화 → 다음 컬럼 문자열 누적 준비

문자 누적

```
else:
    current += char
```

- 설명:**
 - 쉼표가 아니거나 큰따옴표 안인 경우, `current`에 문자 추가
 - 결국 한 컬럼의 문자열이 완성될 때까지 `current`에 누적

마지막 컬럼 처리

```
row.append(current)
```

- 설명:**
 - 반복이 끝난 후 마지막 컬럼은 쉼표 뒤에 있으므로 아직 `row`에 추가되지 않음
 - `current`를 `row`에 추가하여 **모든 컬럼 완성**

반환

```
return row
```

- 설명:**
 - 한 줄을 컬럼 단위로 분리한 리스트 반환
 - 예시:

```
['1', 'Action, Adventure, Drama', '8.9']
```

결과

```
NumPy 배열 형태의 데이터 출력: [['The Shawshank Redemption (1994)' 'Drama' 9.3 1994]
['The Godfather (1972)' 'Crime, Drama' 9.2 1972]
['The Dark Knight (2008)' 'Action, Crime, Drama' 9.0 2008]
...
['JFK (1991)' 'Drama, History, Thriller' 8.0 1991]
['Nights of Cabiria (1957)' 'Drama' 8.1 1957]
['Throne of Blood (1957)' 'Drama, History' 8.1 1957]]
```

데이터 크기(shape) 확인: (1000, 4)

```
NumPy 배열 형태의 데이터 출력: [['The Shawshank Redemption (1994)' 'Drama' 9.3 1994]
['The Godfather (1972)' 'Crime, Drama' 9.2 1972]
['The Dark Knight (2008)' 'Action, Crime, Drama' 9.0 2008]
...
['JFK (1991)' 'Drama, History, Thriller' 8.0 1991]
['Nights of Cabiria (1957)' 'Drama' 8.1 1957]
['Throne of Blood (1957)' 'Drama, History' 8.1 1957]]
```

데이터 크기(shape) 확인: (1000, 4)

종료 코드 0(으)로 완료된 프로세스

Step 2: 기본적인 데이터 탐색

목표

- Step1에서 만든 `data_array.npy` 불러오기
 - 전체 영화 개수 확인
 - 영화 평점(`Rate`) 관련 통계 계산
 - 평균 평점
 - 최고 평점
 - 최저 평점
 - NumPy 배열 형태와 일부 데이터를 확인
-

접근 방법

1. NumPy 배열 불러오기

- `.npy` 파일은 NumPy 전용 바이너리 파일
- `allow_pickle=True` 옵션 필요 (`dtype=object` 배열)

2. 데이터 구조 확인

- 배열 일부 출력 및 `.shape` 확인

3. 전체 영화 개수

- 배열의 행 수 = 영화 개수

4. 평점(Rate) 통계 계산

- 3번째 컬럼(`Rate`, 인덱스 2) 추출
 - `float` 타입 변환
 - 평균, 최고, 최저 계산
-

구현 코드

```
import numpy as np

# NumPy 배열 불러오기
data_array = np.load("../04_outputs/data_array.npy", allow_pickle=True)

# 총 영화 개수
total_movies = data_array.shape[0] # NumPy 배열 행 수 = 영화 개수
print("총 영화 개수:", total_movies)

# 평점 (Rate) 관련 통계
# 평점은 배열의 3번째 컬럼 (인덱스 2)
ratings = data_array[:, 2].astype(float) # 평점 컬럼 추출 및 float 변환
ave_rat = np.mean(ratings) # 평점
max_rat = np.max(ratings) # 최고
min_rat = np.min(ratings) # 최저

print(f"평균 평점: {ave_rat:.2f}\n최고 평점: {max_rat}\n최저 평점: {min_rat}")
```

결과

총 영화 개수: 1000
평균 평점: 8.10
최고 평점: 9.3
최저 평점: 8.0

총 영화 개수: 1000
평균 평점: 8.1
최고 평점: 9.3
최저 평점: 8.0

종료 코드 0(으)로 완료된 프로세스

1. `np.load(..., allow_pickle=True)`
 - `dtype=object` 배열을 불러올 때 필요
 - 문자열과 숫자가 혼합된 배열에 대응
2. 전체 영화 개수
 - `.shape[0]` = 배열 행 수
3. 평점 컬럼 추출 및 타입 변환
 - `data_array[:, 2]` : → 모든 행 선택 2 → 2번째 열 선택 (0부터 시작, 즉 세 번째 컬럼)
즉, `data_array[:, 2]` 는 모든 영화의 평점(Rate) 컬럼만 추출한 1차원 배열.
 - 문자열 형태로 저장되어 있을 수 있으므로 NumPy 배열의 자료형을 강제로 변환하는 메서드 `astype(float)` 필요
4. 평점 통계 계산
 - `np.mean()`, `np.max()`, `np.min()` 사용
 - 소수점 출력은 `.2f` 으로 깔끔하게 조절 가능

Step3_평점이 높은 영화 찾기

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("/content/IMDB_Top_1000.csv")

data_array = df.to_numpy()

titles = data_array[:, 0]
rates = data_array[:, 2].astype(float)
years = data_array[:, 3].astype(int)

max_rate = rates.max()
print("최고 평점:", max_rate)

mask = (rates == max_rate)
print(mask[:10])
```

Step3_평점이 높은 영화 찾기

```
top_titles = titles[mask]
top_rates = rates[mask]

print("★최고 평점 영화 목록:\n")

for t, r in zip(top_titles, top_rates):
    print(f"{t} - 평점: {r:.1f}")

plt.figure(figsize=(10, 4))
x_index = range(len(top_titles))

plt.bar(x_index, top_rates)
plt.title("최고 평점 영화들")
plt.ylabel("평점")

plt.xticks(x_index, top_titles, rotation=45, ha="right")

plt.tight_layout()
plt.show()
```

Step4_장르별 평균 평점 분석

```
import matplotlib.pyplot as plt

genre_counts = {genre: 0 for genre in unique_genres}

for s in genres:
    for g in s.split(","):
        g = g.strip()
        if g in genre_counts:
            genre_counts[g] += 1

sorted_items = sorted(genre_counts.items(), key=lambda x: x[1], reverse=True)

sorted_genres = [g for g, c in sorted_items]
sorted_counts = [c for g, c in sorted_items]
```

Step4_장르별 평균 평점 분석

```
plt.figure(figsize=(12, 6))
plt.bar(sorted_genres, sorted_counts)
plt.xticks(rotation=45, ha='right')

plt.xlabel("Genre")
plt.ylabel("Number of Movies")
plt.title("Number of Movies per Genre")
plt.tight_layout()
plt.show()
```

Step5_연도별 평점 변화 분석

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
years = data_array[:, 3].astype(int)
```

```
rates = data_array[:, 2].astype(float)
```

```
unique_years = np.unique(years)
```

```
year_stats = []
```

Step5_연도별 평점 변화 분석

```
for y in unique_years:

    mask = (years == y)

    year_rates = rates[mask]

    mean_y = year_rates.mean()

    count_y = year_rates.size

    year_stats.append((y, mean_y, count_y))

year_stats_sorted = sorted(year_stats, key=lambda x: x[0])
```

Step5_연도별 평점 변화 분석

```
print("연도별 평균 평점(연도 오름차순):\n")

for y, mean_y, count_y in year_stats_sorted:

    print(f"{y} - {mean_y:.1f}")

plot_years = [y for y, mean_y, count_y in year_stats_sorted]

plot_means = [mean_y for y, mean_y, count_y in year_stats_sorted]

plt.figure(figsize=(12, 6))

plt.plot(plot_years, plot_means, marker='o')

plt.xlabel("Year")

plt.ylabel("Average Rating")

plt.title("Average Movie Rating by Year")

plt.grid(True)

plt.tight_layout()

plt.show()
```

프로젝트 확장

- 특정 감독(Director)별 평균 평점 분석
- 특정 배우(Star) 별 평균 평점 비교
- 특정 조건(예: 평점 8.0 이상 & 2000년 이후 제작) 충족하는 영화만 필터링

확장 prepare:

- 기존 Step1 데이터를 기반으로 프로젝트 확장 분석용 컬럼 포함
- Director, Star 컬럼을 추가로 선택하고, 기존 컬럼(Title, Genre, Rate)과 함께 NumPy 배열로 변환

기존 Step1과 다른 점 / 확장점

항목	기존 Step1	확장 Step1
컬럼 선택	Title, Genre, Rate	Title, Genre, Rate + Director, Star
Released_Year 처리	기존과 동일	주석 명시, int 변환
결측값 제거	기존 컬럼만	확장 컬럼 포함
저장 파일	data_array.npy	data_array_extension.npy
활용	Step2~Step5 분석	감독/배우/조건 필터링 분석 가능

처리 순서

- CSV 파일 읽기 (IMDB Top 1000.csv)
- 큰 따옴표 안/밖 쉼표 구분 파싱 (parse_csv_line)
- 필요한 컬럼 선택:
 - 기존: Title, Genre, Rate
 - 확장: Director, Star 추가 선택
- Title 전처리
 - 앞 숫자 제거
 - 괄호 안 연도 추출 (Released_Year)
- 결측값 제거
- 데이터 타입 변환
 - Rate → float
 - Released_Year → int
- NumPy 배열 변환 (dtype=object)
- 확장용 .npy 저장 (data_array_extension.npy)

코드

```
import numpy as np
```

```
# csv 파일 경로
csv_path = "../01_data/IMDB top 1000.csv"
```

```
# csv 파싱 함수 정의
# 큰 따옴표 안에 있는 쌍표와 바깥에 있는 쌍표 구분
def parse_csv_line(line):
    row = []      # 한 줄을 분리한 문자열 담을 리스트
    current = ""  # 현재 문자열 누적
    inside_quotes = False # 큰 따옴표 안에 있는지 여부 체크
    for char in line:
        if char == '"' and not inside_quotes: # 큰 따옴표 시작
            inside_quotes = True
        elif char == '"' and inside_quotes: # 큰 따옴표 종료
            inside_quotes = False
        elif char == ',' and not inside_quotes: # 따옴표 밖에 쌍표
            row.append(current) # 여태 누적된 문자열 리스트 안에 담기
            current = ""      # 누적 초기화
        else:
            current += char    # 문자 누적
    row.append(current)      # 마지막 문자열 리스트 안에 담기
    return row              # for문 밖에서 반환
```

```
# csv 파일 읽기
with open(csv_path,"r", encoding="utf-8") as f:
    lines = f.read().splitlines() # 줄 단위 리스트 생성
```

```
# 헤더와 데이터 분리 후
header = parse_csv_line(lines[0]) # 파싱 헤더 추출
data_lines = lines[1:]           # 데이터 줄만 따로 저장
```

```
# -----
# 기존 Step1에서 선택된 컬럼 + 확장용 컬럼 추가
# -----
title_idx = header.index("Title") # 1
genre_idx = header.index("Genre") # 4
rate_idx = header.index("Rate")   # 5
cast_idx = header.index("Cast")   # 8 → 확장 Step1: 감독/배우 정보 파싱용
```

```
# 데이터 전처리 리스트 준비
processed_data = [] # 최종 NumPy 배열로 변환할 데이터 담을 리스트
```

```
# 데이터 줄 반복 처리
for line in data_lines:
    row = parse_csv_line(line) # 파싱하여 컬럼 분리
```

```
# row 길이가 필요한 컬럼 중 가장 큰 인덱스보다 작으면 해당 컬럼이 없다는 의미
if len(row) <= max(title_idx, genre_idx, rate_idx):
    continue # 없다면, 건너 뛴다. 다음 줄을 처리하러 for문 처음으로 돌아감.
```

```
# 컬럼 값 가져오기 및 공백 제거
Num_title = row[title_idx].strip()
genre = row[genre_idx].strip()
rate = row[rate_idx].strip()
```

```
# 확장 step1: Cast 컬럼에서 Direction/Stars 추출
cast_info = row[cast_idx].strip()
director = []
stars = []
if "Director:" in cast_info and "Stars:" in cast_info:
    # Director 추출
    director_str = cast_info.split("Director:")[1].split("| Stars:")[0].strip()
    # Stars 추출
    stars_str = cast_info.split("| Stars:")[1].strip()
```

```
# ',' 기준으로 리스트 분리 + 공백 제거
directors = [d.strip() for d in director_str.split(",") if d.strip()]
stars = [s.strip() for s in stars_str.split(",") if s.strip()]
```

```
# Title 앞에 있는 숫자 제거
title = Num_title.split(':')[1].strip() if ':' in Num_title else Num_title
```

```
# 결측값 제거 : 빈 문자열이 있으면 해당 행 제외
if "" in [title, genre, rate] or len(directors) == 0 or len(stars) == 0:
    continue
```

```
# Title에서 Released_Year 추출
# Title 마지막에 () 형태가 있어야만 슬라이싱
released_year = None
if '(' in title and title[-1] == ')':
    released_year = title[title.rfind('(')+1 : title.rfind(')')]
else:
    # 괄호가 없으면 임의로 None 처리
    released_year = None
```

```
# 타입 변환 후 리스트에 추가
try:
    processed_data.append([title, genre, float(rate), int(released_year), directors, stars])
except ValueError:
    # 변환 불가능한 값이 있으면 건너뛰기
    continue
```

```
# NumPy 배열로 변환
# dtype=object 사용 : 문자열 + 숫자가 혼합된 배열
data_array = np.array(processed_data, dtype=object)
```

```
# 결과 확인
print("NumPy 배열 형태의 데이터 출력: ", data_array)
print("\n데이터 크기(shape) 확인: ", data_array.shape)
```

```
# NumPy 배열 바이너리 파일로 저장
np.save("../03_outputs/results/data_array_extension.npy", data_array)
```

결과

```
배열 형태의 데이터 출력: [['The Shawshank Redemption (1994)' 'Drama' 9.3 1994
list(['Frank Darabont'])
list(['Tim Robbins', 'Morgan Freeman', 'Bob Gunton', 'William Sadler'])]
['The Godfather (1972)' 'Crime, Drama' 9.2 1972
list(['Francis Ford Coppola'])
list(['Marlon Brando', 'Al Pacino', 'James Caan', 'Diane Keaton'])]
['The Dark Knight (2008)' 'Action, Crime, Drama' 9.0 2008
list(['Christopher Nolan'])
list(['Christian Bale', 'Heath Ledger', 'Aaron Eckhart', 'Michael Caine'])]
...
['JFK (1991)' 'Drama, History, Thriller' 8.0 1991 list(['Oliver Stone'])
list(['Kevin Costner', 'Gary Oldman', 'Jack Lemmon', 'Walter Matthau'])]
['Nights of Cabiria (1957)' 'Drama' 8.1 1957 list(['Federico Fellini'])
list(['Giulietta Masina', 'François Périer', 'Franca Marzi', 'Dorian Gray'])]
['Throne of Blood (1957)' 'Drama, History' 8.1 1957
list(['Akira Kurosawa'])
list(['Toshirô Mifune', 'Minoru Chiaki', 'Isuzu Yamada', 'Takashi Shimura'])]
```

데이터 크기(shape) 확인: (887, 6)

```
numpy 배열 형태의 데이터 출력: [['The Shawshank Redemption (1994)' 'Drama' 9.3 1994
list(['Frank Darabont'])
list(['Tim Robbins', 'Morgan Freeman', 'Bob Gunton', 'William Sadler'])]
['The Godfather (1972)' 'Crime, Drama' 9.2 1972
list(['Francis Ford Coppola'])
list(['Marlon Brando', 'Al Pacino', 'James Caan', 'Diane Keaton'])]
['The Dark Knight (2008)' 'Action, Crime, Drama' 9.0 2008
list(['Christopher Nolan'])
list(['Christian Bale', 'Heath Ledger', 'Aaron Eckhart', 'Michael Caine'])]
...
['JFK (1991)' 'Drama, History, Thriller' 8.0 1991 list(['Oliver Stone'])
list(['Kevin Costner', 'Gary Oldman', 'Jack Lemmon', 'Walter Matthau'])]
['Nights of Cabiria (1957)' 'Drama' 8.1 1957 list(['Federico Fellini'])
list(['Giulietta Masina', 'François Périer', 'Franca Marzi', 'Dorian Gray'])]
['Throne of Blood (1957)' 'Drama, History' 8.1 1957
list(['Akira Kurosawa'])
list(['Toshirô Mifune', 'Minoru Chiaki', 'Isuzu Yamada', 'Takashi Shimura'])]
```

데이터 크기(shape) 확인: (887, 6)

중요 코드 0(으)로 완료된 프로세스

확장 Step1: 특정 감독별 평균 평점

```
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import numpy as np

# =====
# 한글 폰트 설정
# =====
font_path = "C:/Windows/Fonts/malgun.ttf" # 맑은 고딕
fontprop = fm.FontProperties(fname=font_path)
plt.rcParams['font.family'] = fontprop.get_name()
plt.rcParams['axes.unicode_minus'] = False

# =====
# 데이터 로드
# =====
data_array = np.load("../03_outputs/results/data_array_extension.npy", allow_pickle=True)

# =====
```

```
# 감독별 평점 수집
# =====
director_ratings = {}
for row in data_array:
    rate = row[2]
    for d in row[4]:
        director_ratings.setdefault(d, []).append(rate)

# =====
# 감독별 평균 평점 계산
# =====
director_avg = {d: round(sum(r)/len(r), 2) for d, r in director_ratings.items()}

# =====
# 정렬 및 구간 나누기
# =====
director_sorted = sorted(director_avg.items(), key=lambda x: x[1], reverse=True)
top10 = director_sorted[:10]
middle10 = director_sorted[len(director_sorted)//2 - 5 : len(director_sorted)//2 + 5]
bottom10 = director_sorted[-10:]
all_scores = [v for v in director_avg.values()]

# 시각화 준비 함수
def prepare(items):
    names = [x[0] for x in items][::-1]
    scores = [x[1] for x in items][::-1]
    return names, scores

top_names, top_scores = prepare(top10)
mid_names, mid_scores = prepare(middle10)
low_names, low_scores = prepare(bottom10)
```

```
===== 상위 10명 =====
Frank Darabont - 평균 평점: 8.95
Francis Ford Coppola - 평균 평점: 8.87
Peter Jackson - 평균 평점: 8.8
Thomas Kail - 평균 평점: 8.7
Irvin Kershner - 평균 평점: 8.7
Robert Zemeckis - 평균 평점: 8.65
Roberto Benigni - 평균 평점: 8.6
Jonathan Demme - 평균 평점: 8.6
George Lucas - 평균 평점: 8.6
Masaki Kobayashi - 평균 평점: 8.6
```

```
===== 중위 10명 =====
Damián Szifron - 평균 평점: 8.1
Nuri Bilge Ceylan - 평균 평점: 8.1
Zoya Akhtar - 평균 평점: 8.1
Umesh Shukla - 평균 평점: 8.1
Wes Anderson - 평균 평점: 8.1
Mamoru Hosoda - 평균 평점: 8.1
Anurag Basu - 평균 평점: 8.1
Steve McQueen - 평균 평점: 8.1
James Mangold - 평균 평점: 8.1
Tom McCarthy - 평균 평점: 8.1
```

```
===== 하위 10명 =====
Walter Salles - 평균 평점: 8.0
Thomas Jahn - 평균 평점: 8.0
Billy Bob Thornton - 평균 평점: 8.0
Mike Leigh - 평균 평점: 8.0
Terry Gilliam - 평균 평점: 8.0
Mamoru Oshii - 평균 평점: 8.0
Henry Selick - 평균 평점: 8.0
Harold Ramis - 평균 평점: 8.0
Taylor Hackford - 평균 평점: 8.0
Martin Brest - 평균 평점: 8.0
```

종료 코드 0(으)로 완료된 프로세스

시각화 코드

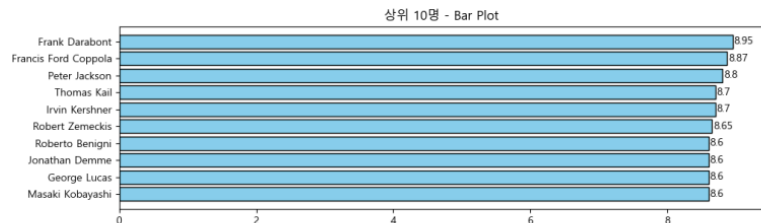
```
# =====  
# 1. 막대 그래프 (한 fig)  
# =====  
fig, axes = plt.subplots(3, 1, figsize=(10, 15))  
fig.suptitle("감독별 평균 평점 - Bar Plot", fontsize=18)  
  
# 상위 10명  
axes[0].barh(top_names, top_scores, color='skyblue', edgecolor='black')  
for x, y in zip(top_scores, top_names):  
    axes[0].text(x + 0.02, y, x, va='center')  
axes[0].set_title("상위 10명 - Bar Plot")  
  
# 중위 10명  
axes[1].barh(mid_names, mid_scores, color='lightgreen', edgecolor='black')  
for x, y in zip(mid_scores, mid_names):  
    axes[1].text(x + 0.02, y, x, va='center')  
axes[1].set_title("중위 10명 - Bar Plot")  
  
# 하위 10명  
axes[2].barh(low_names, low_scores, color='salmon', edgecolor='black')  
for x, y in zip(low_scores, low_names):  
    axes[2].text(x + 0.02, y, x, va='center')  
axes[2].set_title("하위 10명 - Bar Plot")  
  
plt.tight_layout(rect=[0, 0, 1, 0.96])  
plt.show()
```

```
# =====  
# 2. 도트 그래프 (한 fig)  
# =====  
fig, axes = plt.subplots(3, 1, figsize=(10, 15))  
fig.suptitle("감독별 평균 평점 - Dot Plot", fontsize=18)  
  
# 상위 10명  
axes[0].scatter(top_scores, top_names, s=120, color='skyblue')  
for x, y in zip(top_scores, top_names):  
    axes[0].text(x + 0.02, y, x, va='center')  
axes[0].set_title("상위 10명 - Dot Plot")  
  
# 중위 10명  
axes[1].scatter(mid_scores, mid_names, s=120, color='lightgreen')  
for x, y in zip(mid_scores, mid_names):  
    axes[1].text(x + 0.02, y, x, va='center')  
axes[1].set_title("중위 10명 - Dot Plot")  
  
# 하위 10명  
axes[2].scatter(low_scores, low_names, s=120, color='salmon')  
for x, y in zip(low_scores, low_names):  
    axes[2].text(x + 0.02, y, x, va='center')  
axes[2].set_title("하위 10명 - Dot Plot")  
  
plt.tight_layout(rect=[0, 0, 1, 0.96])  
plt.show()
```

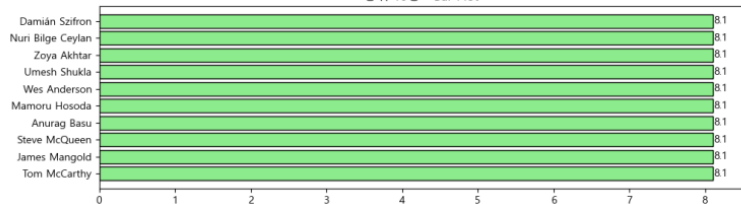
```
# =====  
# 3. 박스 플롯 (전체 데이터)  
# =====  
plt.figure(figsize=(12, 4))  
plt.boxplot(all_scores, vert=False, patch_artist=True,  
            boxprops=dict(facecolor='lightgray', color='black'),  
            medianprops=dict(color='red', linewidth=2),  
            whiskerprops=dict(color='black'),  
            capprops=dict(color='black'))  
plt.title("전체 감독 평균 평점 분포 - Box Plot")  
plt.xlabel("평점")  
plt.yticks([])  
plt.show()
```

결과

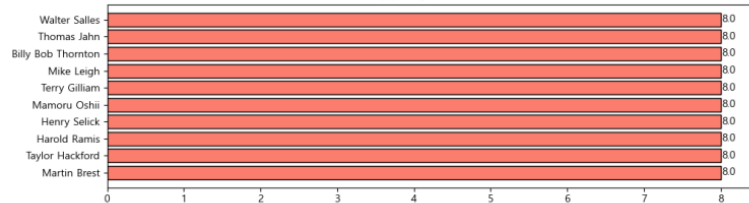
감독별 평균 평점 - Bar Plot



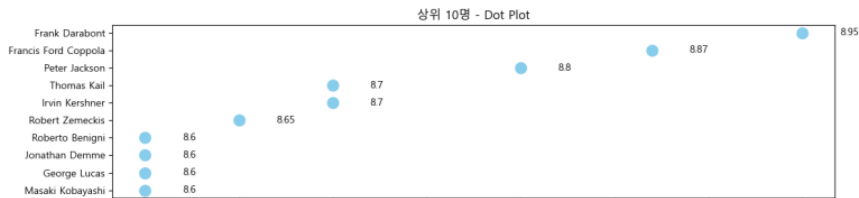
중위 10명 - Bar Plot



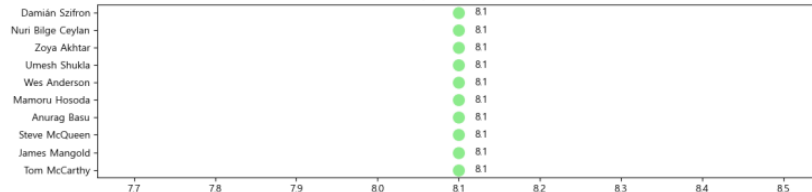
하위 10명 - Bar Plot



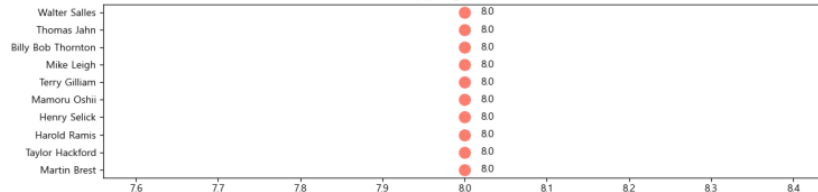
감독별 평균 평점 - Dot Plot



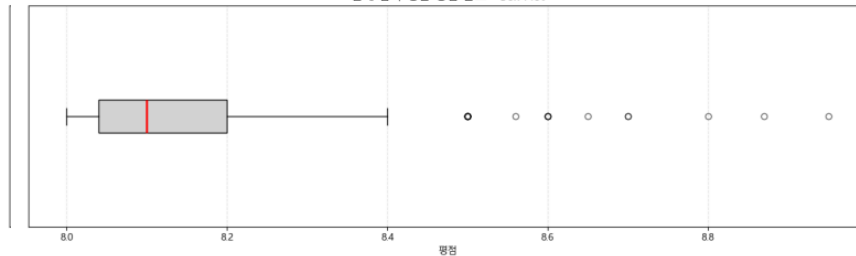
중위 10명 - Dot Plot



하위 10명 - Dot Plot



전체 감독 평균 평점 분포 - Box Plot



확장 Step2: 특정 배우별 평균 평점

```
# =====  
# 데이터 로드  
# =====  
data_array = np.load("../03_outputs/results/data_array_extension.npy", allow_pickle=True)  
  
# =====  
# 배우별 평점 수집  
# =====  
star_ratings = {}  
for row in data_array:  
    rate = row[2]  
    for star in row[5]:  
        star_ratings.setdefault(star, []).append(rate)  
  
# =====  
# 배우별 평균 평점 계산  
# =====  
star_avg = {s: round(sum(r)/len(r), 2) for s, r in star_ratings.items()}  
  
# =====  
# 정렬 및 구간 나누기  
# =====  
star_sorted = sorted(star_avg.items(), key=lambda x: x[1], reverse=True)  
top10 = star_sorted[:10]  
middle10 = star_sorted[len(star_sorted)//2 - 5 : len(star_sorted)//2 + 5]  
bottom10 = star_sorted[-10:]  
all_scores = [v for v in star_avg.values()]  
  
# =====  
# 콘솔 출력  
# =====  
print("\n===== 상위 10명 =====")  
for n, s in top10:  
    print(f"{n} - 평균 평점: {s}")  
  
print("\n===== 중위 10명 =====")  
for n, s in middle10:  
    print(f"{n} - 평균 평점: {s}")  
  
print("\n===== 하위 10명 =====")  
for n, s in bottom10:  
    print(f"{n} - 평균 평점: {s}")
```

```
===== 상위 10명 =====  
Tim Robbins - 평균 평점: 9.3  
Bob Gunton - 평균 평점: 9.3  
William Sadler - 평균 평점: 9.3  
Diane Keaton - 평균 평점: 9.1  
Heath Ledger - 평균 평점: 9.0  
Aaron Eckhart - 평균 평점: 9.0  
John Travolta - 평균 평점: 8.9  
Caroline Goodall - 평균 평점: 8.9  
Martin Balsam - 평균 평점: 8.9  
John Fiedler - 평균 평점: 8.9
```

```
===== 중위 10명 =====  
Bárbara Lennie - 평균 평점: 8.1  
Min-hee Kim - 평균 평점: 8.1  
Jung-woo Ha - 평균 평점: 8.1  
Jin-woong Cho - 평균 평점: 8.1  
So-ri Moon - 평균 평점: 8.1  
Anne Dorval - 평균 평점: 8.1  
Antoine Olivier Pilon - 평균 평점: 8.1  
Suzanne Clément - 평균 평점: 8.1  
Patrick Huard - 평균 평점: 8.1  
Shahid Kapoor - 평균 평점: 8.1
```

```
===== 하위 10명 =====  
Stephen Tobolowsky - 평균 평점: 8.0  
Damian Chapa - 평균 평점: 8.0  
Jesse Borrego - 평균 평점: 8.0  
Benjamin Bratt - 평균 평점: 8.0  
Enrique Castillo - 평균 평점: 8.0  
Chris O'Donnell - 평균 평점: 8.0  
James Rebhorn - 평균 평점: 8.0  
Gabrielle Anwar - 평균 평점: 8.0  
Kevin Costner - 평균 평점: 8.0  
Walter Matthau - 평균 평점: 8.0
```

종료 코드 0(으)로 완료된 프로세스

시각화 코드

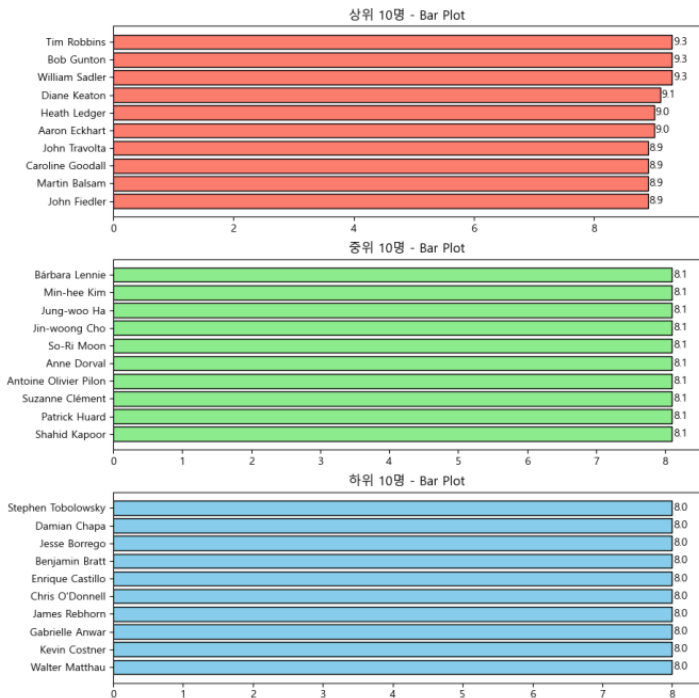
```
# =====  
# 시각화 준비 함수  
# =====  
def prepare(items):  
    names = [x[0] for x in items][:-1]  
    scores = [x[1] for x in items][:-1]  
    return names, scores  
  
top_names, top_scores = prepare(top10)  
mid_names, mid_scores = prepare(middle10)  
low_names, low_scores = prepare(bottom10)  
  
# =====  
# 1. 막대 그래프 (상위·중위·하위)  
# =====  
fig, axes = plt.subplots(3, 1, figsize=(10, 15))  
fig.suptitle("배우별 평균 평점 - Bar Plot", fontsize=18)  
  
axes[0].barh(top_names, top_scores, color='salmon', edgecolor='black')  
for x, y in zip(top_scores, top_names):  
    axes[0].text(x + 0.02, y, x, va='center')  
axes[0].set_title("상위 10명 - Bar Plot")  
  
axes[1].barh(mid_names, mid_scores, color='lightgreen', edgecolor='black')  
for x, y in zip(mid_scores, mid_names):  
    axes[1].text(x + 0.02, y, x, va='center')  
axes[1].set_title("중위 10명 - Bar Plot")  
  
axes[2].barh(low_names, low_scores, color='skyblue', edgecolor='black')  
for x, y in zip(low_scores, low_names):  
    axes[2].text(x + 0.02, y, x, va='center')  
axes[2].set_title("하위 10명 - Bar Plot")  
  
plt.tight_layout(rect=[0, 0, 1, 0.96])  
plt.show()
```

```
# =====  
# 2. 도트 그래프 (상위·중위·하위)  
# =====  
fig, axes = plt.subplots(3, 1, figsize=(10, 15))  
fig.suptitle("배우별 평균 평점 - Dot Plot", fontsize=18)  
  
axes[0].scatter(top_scores, top_names, s=120, color='salmon')  
for x, y in zip(top_scores, top_names):  
    axes[0].text(x + 0.02, y, x, va='center')  
axes[0].set_title("상위 10명 - Dot Plot")  
  
axes[1].scatter(mid_scores, mid_names, s=120, color='lightgreen')  
for x, y in zip(mid_scores, mid_names):  
    axes[1].text(x + 0.02, y, x, va='center')  
axes[1].set_title("중위 10명 - Dot Plot")  
  
axes[2].scatter(low_scores, low_names, s=120, color='skyblue')  
for x, y in zip(low_scores, low_names):  
    axes[2].text(x + 0.02, y, x, va='center')  
axes[2].set_title("하위 10명 - Dot Plot")  
  
plt.tight_layout(rect=[0, 0, 1, 0.96])  
plt.show()
```

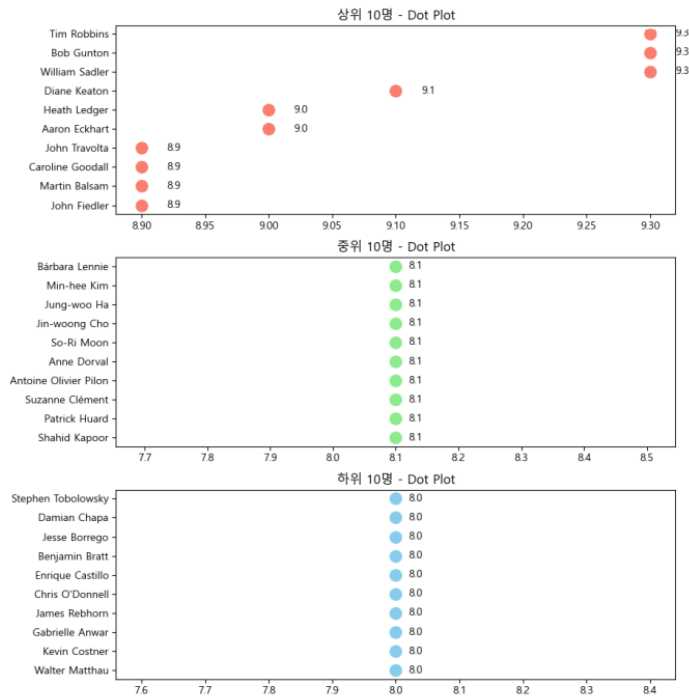
```
# =====  
# 3. 박스 플롯 (전체 데이터)  
# =====  
plt.figure(figsize=(12, 4))  
plt.boxplot(all_scores, vert=False, patch_artist=True,  
            boxprops=dict(facecolor='lightgray', color='black'),  
            medianprops=dict(color='red', linewidth=2),  
            whiskerprops=dict(color='black'),  
            capprops=dict(color='black'),  
            flierprops=dict(marker='o', color='orange', alpha=0.6))  
  
plt.title("전체 배우 평균 평점 분포 - Box Plot")  
plt.xlabel("평점")  
plt.yticks([])  
plt.grid(axis='x', linestyle='--', alpha=0.3)  
plt.tight_layout()  
plt.show()
```

결과

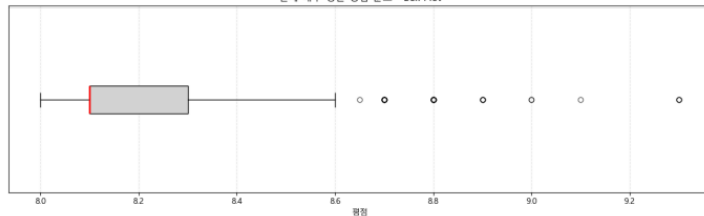
배우별 평균 평점 - Bar Plot



배우별 평균 평점 - Dot Plot



전체 배우 평균 평점 분포 - Box Plot



확장 Step3: 특정 조건 충족하는 영화만 필터링

```
# =====  
# 데이터 로드 및 필터링 (평점 8.0 이상, 2000년 이후 제작 영화)  
# =====  
data_array = np.load("../03_outputs/results/data_array_extension.npy", allow_pickle=True)  
  
min_rate = 8.0  
min_year = 2000  
  
filtered = []  
for row in data_array:  
    title, rate, year = row[0], row[2], row[3]  
    if rate >= min_rate and year >= min_year:  
        filtered.append([title, rate, year])  
  
filtered_array = np.array(filtered, dtype=object)  
  
# =====  
# 출력: 총 영화 수 및 상위 10개 영화 정보  
# =====  
print(f"총 영화 수 (평점 {min_rate} 이상 & {min_year}년 이후 제작): {len(filtered_array)}\n")  
print("상위 10개 영화 (평점 높은 순):")  
# 평점 기준 내림차순 정렬  
top10_movies = sorted(filtered_array, key=lambda x: x[1], reverse=True)[:10]  
for movie in top10_movies:  
    print(f"{movie[0]} - 평점: {movie[1]}")
```

총 영화 수 (평점 8.0 이상 & 2000년 이후 제작): 496

상위 10개 영화 (평점 높은 순):

The Dark Knight (2008) - 평점: 9.0

The Lord of the Rings: The Return of the King (2003) - 평점: 8.9

Inception (2010) - 평점: 8.8

The Lord of the Rings: The Fellowship of the Ring (2001) - 평점: 8.8

Hamilton (2020) - 평점: 8.7

The Lord of the Rings: The Two Towers (2002) - 평점: 8.7

Parasite (2019) - 평점: 8.6

Interstellar (2014) - 평점: 8.6

Spirited Away (2001) - 평점: 8.6

Joker (2019) - 평점: 8.5

종료 코드 0(으)로 완료된 프로세스

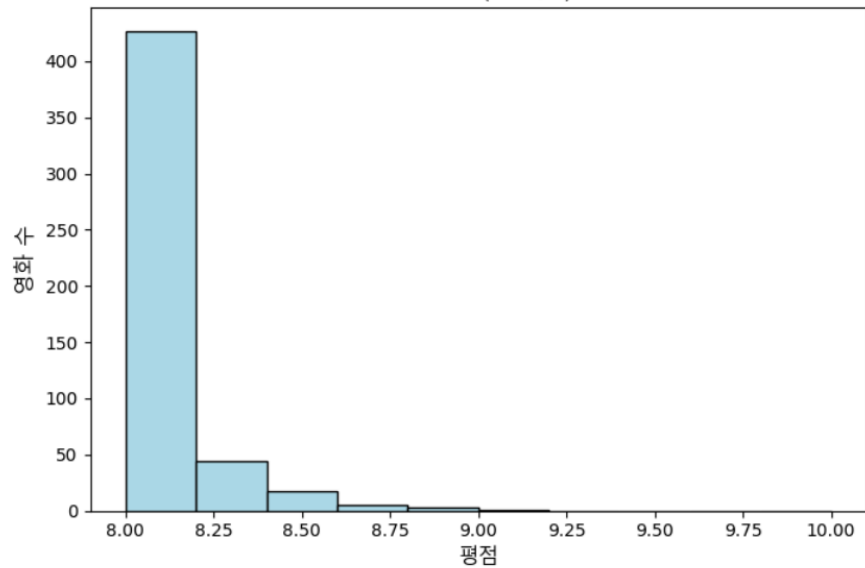
시각화 코드

```
# =====  
# Dot Plot & 히스토그램을 위한 데이터 준비  
# =====  
rates = filtered_array[:,1].astype(float)  
years = filtered_array[:,2].astype(int)  
  
unique_years = np.unique(years)  
avg_rates = []  
movie_counts = []  
  
for y in unique_years:  
    y_rates = rates[years == y]  
    avg_rates.append(np.mean(y_rates))  
    movie_counts.append(len(y_rates))  
  
sizes = [c*20 for c in movie_counts] # 점 크기 조정
```

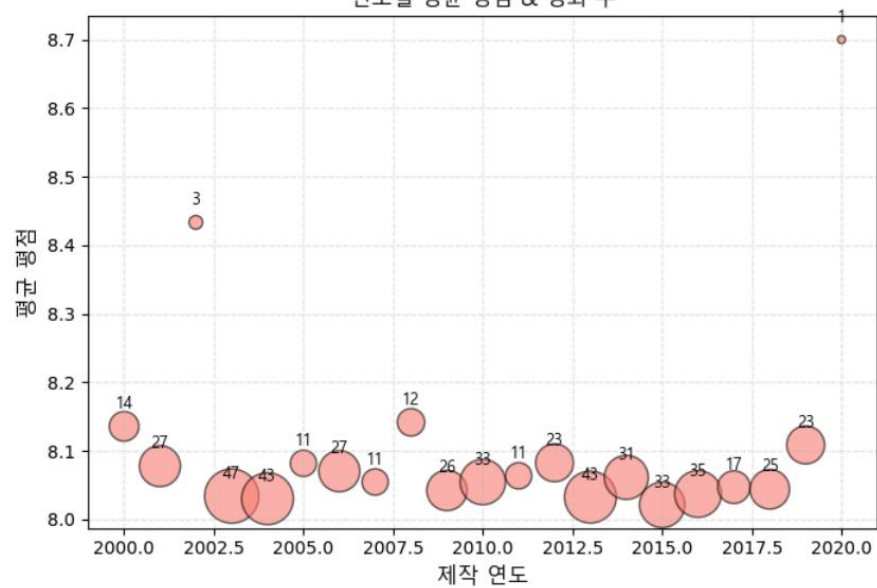
```
# =====  
# 시각화  
# =====  
fig, axes = plt.subplots(1, 2, figsize=(14,5)) # 서브플롯 2개  
  
# 1) 히스토그램 - 평점 분포  
axes[0].hist(rates, bins=np.arange(8, 10.1, 0.2), color='lightblue', edgecolor='black')  
axes[0].set_xlabel("평점", fontproperties=fontprop)  
axes[0].set_ylabel("영화 수", fontproperties=fontprop)  
axes[0].set_title("평점 분포 (8.0 이상)", fontproperties=fontprop)  
  
# 2) 연도별 평균 평점 & 영화 수 Dot Plot  
axes[1].scatter(unique_years, avg_rates, s=sizes, color='salmon', alpha=0.6, edgecolor='black')  
for x, y, c in zip(unique_years, avg_rates, movie_counts):  
    axes[1].text(x, y + 0.02, str(c), ha='center', va='bottom', fontproperties=fontprop, fontsize=9)  
axes[1].set_xlabel("제작 연도", fontproperties=fontprop)  
axes[1].set_ylabel("평균 평점", fontproperties=fontprop)  
axes[1].set_title("연도별 평균 평점 & 영화 수", fontproperties=fontprop)  
axes[1].grid(True, linestyle='--', alpha=0.3)  
  
plt.tight_layout()  
plt.show()
```

결과

평점 분포 (8.0 이상)



연도별 평균 평점 & 영화 수



참고 사이트

- IMDb Non-Commercial Datasets : <https://developer.imdb.com/non-commercial-datasets/> (IMDb Developer)
- IMDb Review Data 준비 및 Preprocess : <https://velog.io/@olxtar/IMDb-Review-Data-%EC%A4%80%EB%B9%84-%EB%B0%8F-Preprocess> (velog)
- 10-04 IMDB 리뷰 감성 분류하기(IMDB Movie Review ... : <https://wikidocs.net/24586> (위키독스)